

Vuka: Specification and 48-Hour Build Plan

GovTech 2026 · Shesha · 2026-09-17 · @Someone

1. Scope and architecture

What is being built

A performance reporting system for the Department of Sport, Arts and Culture and the 32 bodies it funds — 26 public entities and six NPOs.

The system has two halves. **A standard:** a machine-readable schema for quarterly performance reporting, derived from the structure entities already use in their Annual Performance Plans, published as a spreadsheet template. **A system:** submission, repository, workspaces, analytics and early warning, built on that spine.

Stack

Layer	Choice	Why
Language	Java 21	Team's strongest language. Records and switch expressions used throughout.
Framework	Spring Boot 3.3	Web, Data JPA, Security, Thymeleaf, Validation, Scheduling.
Database	PostgreSQL 16	The domain is joins. Allocation → Target → TargetResult is the product, and aggregating across it is what SQL is for.
ORM	Hibernate via Spring Data JPA	<code>ddl-auto: validate</code> only — Flyway owns the schema.
Migrations	Flyway	Versioned SQL. The schema is a reviewable artifact, not a side effect.
Identity	Firebase Auth	Sign-up, password reset and token issuance for free. Verified server-side.
Spreadsheets	Apache POI	Reads the reporting template, writes the export.

Layer	Choice	Why
Rich UI	React	DSAC dashboard only.
Light UI	Thymeleaf, server-rendered	Mobile submission and citizen view. No framework to download is the only way to hit the payload budget.

Why Postgres and not a document store

Every differentiating feature is a query across relationships. Unit cost joins allocation to target to result. The risk engine aggregates submissions, targets, results, allocations and audit findings per entity. A document store would make both awkward and neither faster.

It also makes the sovereignty claim true rather than rhetorical: Postgres is open source, portable, and available as managed Cloud SQL in `africa-south1` (Johannesburg) if data residency is required. Nothing in the schema ties the department to a provider.

Identity versus data

Firebase holds **only** who someone is. Every fact about performance lives in Postgres.

A browser signs in against Firebase and receives an ID token carrying two custom claims: `role` and, for entity reporters, `entityId`. A Spring Security filter verifies that token on every request and maps the claims to authorities. The `entityId` claim is the security boundary for the whole system — it sits on a signed token rather than in a request parameter precisely so a client cannot choose it.

Component layout

```

za.gov.dsac.vuka
  config/          Firebase verification, Spring Security, the authenticated
principal
  domain/          17 JPA entities, one enum holder
  repository/      15 Spring Data repositories
  service/
    RiskEngine      pure scoring: no Spring, no JPA, fully unit-tested
    RiskService      gathers inputs, persists scores and signals
    TemplateParser   Apache POI, template -> extractions with source cells
    SubmissionService upload -> parse -> confirm -> submit -> review
    UnitCostService  planned versus actual, sector-bound peer comparison
    NotificationService 30 day / 15 day / hourly countdowns
    PublicationService citizen projection, gated on departmental approval
    SeedService       illustrative data, to be replaced
  web/             REST controllers plus two server-rendered surfaces

```

RiskEngine deliberately has no framework in it. It takes plain numbers and returns plain records, so the scoring is testable without a database, a container or a context. That is what makes the explainability claim checkable rather than asserted.

2. Functional requirements

FR-1 to FR-5 come directly from the challenge statement and are the floor. FR-6 to FR-11 are what differentiate the submission. Each carries a priority: **MUST** ships or there is no submission, **SHOULD** ships unless time runs out, **MAY** is the flourish.

FR-1 · Document repository · MUST

FR-1.1	Entities upload strategic plans, APPs, operational plans, annual reports, quarterly reports and financials.
FR-1.2	Re-uploading the same document type for the same entity creates a new version rather than overwriting.
FR-1.3	Each document carries an approval state, settable only by DSAC.

FR-1.4	DSAC can read every entity's documents; an entity reads only its own.
---------------	---

FR-2 · Analytics · MUST

FR-2.1	Per-entity dashboard showing targets by state: not started, in progress, achieved, deadline missed.
FR-2.2	Year-on-year comparison of achievement against prior financial years.
FR-2.3	Audit findings displayed with repeat findings distinguished from fresh ones.
FR-2.4	Workforce statistics per entity: headcount, jobs created, and demographic aggregates.
FR-2.5	Portfolio view across all 32 bodies, ordered by risk rather than alphabetically .

FR-3 · Early warning · MUST

FR-3.1	Countdown notifications fire at exactly 30 days and 15 days before a regulatory deadline, then hourly within the final day.
FR-3.2	Notifications name which targets currently have no evidence attached, rather than only stating that a report is due.
FR-3.3	An entity that has already submitted is not notified.
FR-3.4	Risk-based prediction of non-performance, drawing on reporting history and audit history, ahead of any deadline being missed.

FR-4 · Workspaces · MUST

FR-4.1	Per-entity workspace bringing together documents, submissions, targets and tasks.
FR-4.2	Comments anchored to a specific target, result or document rather than to the entity generally.
FR-4.3	Comments visible without a page reload.
FR-4.4	Task assignment, internally within DSAC and externally to entity staff.
FR-4.5	Upload receipt is confirmed to the uploader.
FR-4.6	Office document formats read on input and written on export.
FR-4.7	Full functionality on a mobile device.

FR-5 · Security and privacy · MUST

See §3. Stated as a functional requirement in the brief; specified as non-functional here because every item is a constraint on how the system behaves rather than a feature a user invokes.

FR-6 · Structured submission · MUST

FR-6.1	A published reporting template defines the standard: one row per indicator, fixed columns, documented schema.
FR-6.2	An uploaded template is parsed into extraction records, each carrying the sheet and cell it came from.
FR-6.3	Rows that cannot be matched to a registered target are flagged for manual matching, never silently discarded.
FR-6.4	Confidence is recorded per extracted value, lower for numbers typed as free text.

FR-6.5	No extracted value becomes performance data until a named human confirms it.
FR-6.6	Confirmation records the confirming person's identity and timestamp on every figure.

FR-7 · Review · MUST

FR-7.1	DSAC approves a submission, or returns it with a stated reason.
FR-7.2	No role can edit a confirmed figure. A disputed number is corrected by the entity re-confirming, not by a reviewer overwriting.
FR-7.3	Returned submissions reappear in the entity's outstanding list marked for correction.

FR-8 · Explainable risk scoring · SHOULD

FR-8.1	A risk score per entity per reporting period, on a 0 to 100 scale, banded low / medium / high / critical.
FR-8.2	Five contributing signals: submission lateness, evidence gap, spend-to-delivery divergence, prior audit findings, revision churn.
FR-8.3	Each signal stored with its raw value, normalised value, weight, contribution and a plain-language description.
FR-8.4	The score is never displayed without its signals.
FR-8.5	Contributions always sum to the score.
FR-8.6	Scoring is deterministic and rule-weighted. No trained model.
FR-8.7	Movement since the previous computation is shown alongside the level.

FR-9 · Unit cost · SHOULD

FR-9.1	Planned unit cost derived from the APP: allocation behind a target, divided by the annual target.
FR-9.2	Actual unit cost derived from reported spend divided by reported delivery.
FR-9.3	Entity compared against its own plan, as the default view.
FR-9.4	Entity compared against its own prior years.
FR-9.5	Peer comparison only within a matching sector , labelled indicative.
FR-9.6	No interface or API parameter permits comparison across sectors. The absence is the requirement.

FR-10 · Low-bandwidth submission · SHOULD

FR-10.1	A mobile submission path presenting one indicator per screen.
FR-10.2	Progress carried in the URL, not in server session state, so a dropped connection loses nothing.
FR-10.3	Produces identical data to the desktop path, through the same service layer.

FR-11 · Citizen view · SHOULD

FR-11.1	An unauthenticated page per entity: allocation, commitments, delivery, date last reported.
FR-11.2	Visible only for entities DSAC has explicitly approved for publication.
FR-11.3	Carries no risk scores, unconfirmed figures, personal information or internal comments.
FR-11.4	An unpublished entity returns 404, so the public surface never confirms what sits behind it.

FR-12 · Report once · MAY

FR-12.1	Export a submission as a spreadsheet for onward filing to Treasury or the Auditor-General.
----------------	--

FR-13 · Messaging channel · MAY

FR-13.1	Submission over WhatsApp, landing in the same data model through the same service layer.
----------------	--

FR-13.2	Constraint to state honestly: verified WhatsApp Business access takes weeks of review, so a sandbox number is the only path inside the build window.
----------------	--

3. Non-functional requirements

NFR-1 · Access control

NFR-1.1	Four roles: ENTITY_REPORTER , DSAC_REVIEWER , DSAC_EXECUTIVE , ADMIN .
NFR-1.2	Role and entity binding travel as signed Firebase custom claims, verified server-side on every request.
NFR-1.3	An entity reporter can read and write only its own entity's data. Enforced from the token claim, never from a request parameter.
NFR-1.4	A reviewer reads all entities but cannot alter a confirmed figure .
NFR-1.5	An administrator manages users and configuration but cannot alter performance data .
NFR-1.6	Segregation of duties is structural: no single role can both produce and silently amend performance information.

NFR-2 · Data protection and POPIA

NFR-2.1	Personal information is limited to reporter contact details. No staff records are held.
NFR-2.2	Workforce demographics are stored and displayed in aggregate only , never at individual level.

NFR-2.3	The public surface exposes no personal information whatsoever.
NFR-2.4	Data residency in South Africa is achievable without code change: Cloud SQL for PostgreSQL is available in <code>africa-south1</code> .
NFR-2.5	Encryption in transit and at rest, delegated to the platform.

NFR-3 · Auditability

NFR-3.1	Every confirmed figure carries the uid, display name and timestamp of the person who confirmed it.
NFR-3.2	Every extracted value retains the document and cell reference it came from.
NFR-3.3	Performance data is append-only. Corrections create new records; originals remain.
NFR-3.4	Any figure in the system can be traced to a person, a date and a source cell.

NFR-4 · Explainability

NFR-4.1	No machine-derived value enters the record without human confirmation. Enforced by schema separation, not by policy.
NFR-4.2	Risk scoring is deterministic and reproducible by hand.
NFR-4.3	Weights are fixed constants, stored with each signal so a historical score stays explainable if weights later change.

NFR-4.4	Band thresholds are round numbers (25 / 50 / 70) so a score can be justified in one sentence.
----------------	---

NFR-5 · Accessibility and bandwidth

NFR-5.1	Mobile submission and citizen pages under 100KB uncompressed . (<i>Achieved: all six templates are under 5KB.</i>)
NFR-5.2	No JavaScript framework on the low-bandwidth surfaces.
NFR-5.3	Response compression enabled for HTML, CSS, JS and JSON.
NFR-5.4	Touch targets no smaller than 44px on the mobile path.
NFR-5.5	Plain language throughout the entity-facing and citizen-facing surfaces. No departmental jargon.
NFR-5.6	Light and dark rendering on every server-rendered page.

NFR-6 · Reliability

NFR-6.1	A dropped connection mid-submission loses no work: progress is in the URL, not in session state.
NFR-6.2	The scheduled scoring job tolerates sparse and null data without failing.
NFR-6.3	Notifications stamp their last fire time so a retry cannot double-send.
NFR-6.4	Schema drift fails loudly at startup (<code>ddl-auto: validate</code>) rather than silently at runtime.

NFR-7 · Sustainability and sovereignty

NFR-7.1	Open-source components throughout. No proprietary runtime dependency.
NFR-7.2	Standard Java and PostgreSQL, so maintenance skills are widely available locally.
NFR-7.3	Deployable on-premise or on any cloud. Nothing is provider-specific except optional Firebase identity, which is replaceable behind the security filter.
NFR-7.4	No per-seat licensing as the number of entities grows.
NFR-7.5	The department can take the codebase and the database in-house.

NFR-8 · Scale

Deliberately modest, and worth stating so nobody over-engineers: **32 entities, roughly 400 targets, four submissions each per year**. That is a few thousand rows a year. The scaling concern is not throughput, it is the number of departments that might adopt the same standard — which the schema supports by adding rows, not tables.

4. Data model

Seventeen entities. The accountability chain is the foreign keys: **Allocation → Target → TargetResult → DocumentRecord / ExtractionResult**. Unit cost and every risk signal are queries along that path.

Reference data

FinancialYear — label (2026/27), startDate, endDate, isCurrent

ReportingPeriod — financialYear, label, quarter (1-4 or null for annual), periodStart, periodEnd, submissionDueDate, regulatoryDeadline

`regulatoryDeadline` is what the countdowns fire against. `submissionDueDate` is what lateness is measured from. They are different dates and conflating them breaks the risk score.

The funded body

PublicEntity — `name`, `shortName`, `entityType` (public entity or NPO), `sector`, `sizeBand`, `mandate`, `contactName`, `contactEmail`, `publiclyVisible`

`publiclyVisible` is the publication switch. It is the only thing standing between an entity's data and the citizen view, and only DSAC can set it.

The chain

Allocation — `entity`, `financialYear`, `programme`, `amount`, `dateApproved`

Target — `entity`, `financialYear`, `allocation`, `indicatorRef`, `outcomeStatement`, `outputStatement`, `indicator`, `unitOfMeasure`, `baseline`, `annualTarget`, `q1Target` ... `q4Target`, `plannedUnitCost`, `revisionCount`

`indicatorRef` is the parser's match key. `plannedUnitCost` is computed at load time ($\text{allocation} \div \text{annual target}$) so the comparison view needs no join. `revisionCount` feeds the churn signal.

Submission — `entity`, `reportingPeriod`, `status`, `channel`, `submittedAt`, `submittedByUid`, `submittedByName`, `reviewedByUid`, `reviewedAt`, `returnReason`, `createdAt`

TargetResult — `target`, `submission`, `indicatorRef`, `actualValue`, `quarterTarget`, `variance`, `varianceExplanation`, `spendToDate`, `actualUnitCost`, `status`, `confirmedByUid`, `confirmedByName`, `confirmedAt`

Confirmed performance data. There is no update path to this table anywhere in the application. `confirmedByUid` is never a service account.

Evidence

DocumentRecord — entity, submission, documentType, fileName, storagePath, sizeBytes, version, uploadedByUid, uploadedAt, approvalStatus

ExtractionResult — submission, documentRecord, target, indicatorRef, fieldName, extractedValue, sourceLocation, confidence, isConfirmed, needsManualMatch

Deliberately a separate table from TargetResult. Machine output lives here with its source cell until a human promotes it. That separation is the human-in-the-loop guarantee, expressed as schema rather than as a paragraph in a policy.

Intelligence

RiskScore — entity, reportingPeriod, score, band, previousScore, signals (one-to-many), computedAt

RiskSignal — riskScore, type, value, normalised, weight, contribution, description

Signals are stored rather than recomputed so that a score from three months ago can still show its working. A recomputed explanation is not an audit record.

AuditFinding — entity, financialYear, outcome, description, isRepeatFinding, resolutionStatus

WorkforceStat — entity, reportingPeriod, headcount, jobsCreated, femaleCount, maleCount, youthCount, disabilityCount

Aggregate counts only. No individual staff record exists anywhere in the schema, which is what keeps the POPIA answer to two sentences.

Workspace

Comment — entity, anchorType (target / result / document), anchorId, parentId, authorUid, authorName, authorRole, body, isResolved, createdAt

TaskItem — entity, submission, assignedToUid, assignedToName, assignedByUid, description, dueDate, status, isExternal

`isExternal` covers the brief's "internally and externally, up and down the operations process" wording.

UserProfile — `uid`, `email`, `displayName`, `role`, `entity`

Reference lists

Java enums persisted as strings, so the database stays readable: `EntityType`, `Sector`, `SizeBand`, `Role`, `SubmissionStatus`, `SubmissionChannel`, `TargetStatus`, `DocumentType`, `ApprovalStatus`, `RiskBand`, `RiskSignalType`, `AuditOutcome`, `ResolutionStatus`, `TaskStatus`, `AnchorType`, `NotificationOffset`.

Indexes

On the paths the dashboard actually walks: `target(entity, financialYear)`, `submission(entity)`, `target_result(submission)`, `extraction_result(submission)`, `risk_score(reportingPeriod, score desc)`, `document_record(entity)`.

5. API surface

All `/api/**` endpoints require a verified Firebase ID token in an `Authorization: Bearer` header. `/public/**` requires nothing.

Entity-facing

Method	Path	Role	Purpose
POST	<code>/api/submissions/open</code>	Reporter	Open or resume a draft for a period
POST	<code>/api/submissions/{id}/upload</code>	Reporter	Upload the template, parse it
GET	<code>/api/submissions/{id}/extractions</code>	Reporter, DSAC	What was read, with source cells
POST	<code>/api/submissions/{id}/confirm</code>	Reporter	Turn extractions into performance data

Method	Path	Role	Purpose
POST	/api/submissions/{id}/submit	Reporter	Hand the period to DSAC

DSAC-facing

Method	Path	Role	Purpose
GET	/api/dashboard/portfolio	DSAC	All 32 bodies, ranked by risk, with signals
GET	/api/dashboard/entity/{id}	DSAC	Risk panel, targets, unit cost, audit history
GET	/api/dashboard/entity/{id}/peers	DSAC	Sector-bound peer comparison
POST	/api/dashboard/recompute	Reviewer, Admin	Force a scoring run
POST	/api/submissions/{id}/review	Reviewer, Admin	Approve, or return with a reason

Server-rendered

Path	Auth	Purpose
/m	Reporter	Outstanding submissions
/m/submission/{id}/step/{n}	Reporter	One indicator per screen
/public	None	Published entities
/public/entity/{id}	None	One entity's citizen record

What is deliberately absent

No endpoint updates a `TargetResult`. Not for a reviewer, not for an administrator. If a figure is wrong, the submission is returned and the entity re-confirms. When someone asks how you prevent a reviewer quietly improving an entity's numbers, the answer is that

there is no code path that would let them.

No parameter permits cross-sector unit cost comparison. `/peers` derives the sector from the entity itself; the caller cannot override it.

6. The 48-hour build

Hours from T0. Team of five. Every block ends somewhere demonstrable, so an overrun costs depth rather than the submission.

The governing rule. Judging criteria 1 and 3 are 40 percent combined and both need things that visibly work. Day one ends with FR-1 through FR-7 present, even if shallow. Nothing in day two starts until that list is green.

Roles, assigned at T0 and not rotated

Role	Owns
Domain	Entities, Flyway migration, repositories, security config. Critical path for eight hours.
Services	Parser, risk engine, notification scheduler, submission lifecycle.
API	Controllers, DTOs, the two Thymeleaf surfaces.
Frontend	React dashboard, risk panel, unit cost view.
Pitch	Real data first, then narrative, demo script, rehearsals. A full role.

H0-H3 · Foundation

Three tracks at once.

Track 1 · Domain and Services: get it running.

```
docker compose up -d
mvn spring-boot:run
curl localhost:8080/public      # no auth needed; if this renders, you are up
```

The provided project already contains the entities, the migration and the seed. **This is a go/no-go gate.** If the application is not serving by H3, stop adding features and spend the time getting a clean `mvn compile`. Expect a handful of import and signature fixes on the first build — the project was written without a reachable Maven Central and has not been compiled end to end.

Track 2 · Pitch and Frontend: real data.

The single highest-leverage block in the build, and it involves no code. Pull from published Annual Performance Plans and annual reports, for four to six entities: name, type, sector; allocation per year for three years; ten to fifteen real targets with indicator wording, unit of measure, baseline, annual target and quarterly split; actuals; Auditor-General outcome per year with repeats marked; headcount and jobs created.

Rewrite `SeedService` around it. **Do not invent numbers.** A risk score over invented history is circular and one question exposes it.

Track 3 · API: Firebase.

Create the project, enable email/password, generate a service account key, set `FIREBASE_CREDENTIALS`, create four users and set their custom claims. Confirm a token round-trips before anyone builds on it.

Exit at H3: application serving, real data in a spreadsheet, four users able to sign in with distinct roles.

H3-H8 · Data and access

- **Domain:** real seed loaded, `plannedUnitCost` computed at load time so the comparison view needs no join later.
- **API:** every controller reachable with a real token; confirm a reporter sees one entity and a reviewer sees all.
- **Services:** `mvn test` green. The risk engine ships with ten tests; they should pass before anything is built on top.
- **Frontend:** React app scaffolded, authenticating, hitting `/api/dashboard/portfolio`.

Exit at H8: portfolio endpoint returns real entities, ranked, with scores.

H8-H16 · The mandatory floor

FR-1 through FR-7 demonstrably present.

Requirement	Owner	Done means
FR-1 Repository	API	Upload, version on re-upload, approval state
FR-2 Analytics	Frontend	Targets by state, year-on-year, audit findings, workforce
FR-3 Early warning	Services	Countdowns firing off real deadlines, naming outstanding targets
FR-4 Workspaces	API + Frontend	Comments anchored to targets, task assignment, 5s refresh
FR-5 Security	Domain	Four roles enforced end to end
FR-6 Submission	Services + API	Upload, parse, confirm, submit
FR-7 Review	API	Approve and return

Two cheap wins. Task assignment is named in the brief and is CRUD plus an assignee picker — half an hour of free relevance. And FR-4.3 is satisfied by a five-second poll; do not build websockets for a demo.

Exit at H16 — the day-one gate. Walk the floor end to end out loud, in front of each other. Log in as a reporter, submit; log in as a reviewer, approve. Fix anything missing **before anyone sleeps**.

H16-H22 · Sleep

Rotate one person on light cover. Everyone else sleeps. This block is in the plan on purpose: a team that skips it writes the bugs it spends H30 to H40 hunting.

H22-H30 · Intelligence

- **Services:** tune the risk weights against the real data. If the ranking contradicts what the annual reports say, the weights are wrong, not the reports. Re-run `mvn test` after every change.
- **Frontend:** the risk panel. Score with its signals expanded beneath it. This one component carries FR-8, NFR-4 and a large share of two judging criteria — give it disproportionate effort.
- **API:** unit cost endpoint returning planned against actual per target.
- **Pitch:** first full rehearsal against whatever exists. It will be rough. Do it anyway.

Exit at H30: open an entity, see a score, expand it, understand why.

H30-H36 · Differentiators

- **Frontend:** unit cost view — planned versus actual as the default, own-history second, sector peers labelled indicative.
- **API:** citizen page live, gated on `publiclyVisible`.
- **Services:** mobile submission path working end to end.
- **Domain:** Excel export of a submission (FR-12).

Exit at H36: the full demo thread runs start to finish, even if rough.

H36-H40 · Conditional

WhatsApp over a sandbox number, **only if H36's exit was clean**. If the webhook is not receiving by H39, drop it and say in the pitch that the channel is next with the mobile view shipping today. Otherwise spend the block running the demo thread ten times — things that work once often fail on the eighth attempt, and the judges are the ninth.

H40 · Feature freeze

Hard stop. No new functionality after this line whatever state it is in. The commonest way a good project loses is a half-finished feature breaking a working demo at hour 46. If it is not done at H40, delete the branch rather than merging it.

H40-H44 · Rehearse and insure

1. Record the demo first. Before anything else in this block. Screen recording, full thread, narrated, end to end. Conference wifi fails and databases lock; a recording costs twenty minutes and is the difference between a bad five minutes and no submission. Keep it on a local drive, not behind a link.

2. Rehearse three times, on the clock. Standing up, out loud, timer running, including the questions afterwards.

Time	Screen	Say
0:00	Phone at <code>/m</code> , one indicator	"An NPO with three staff and no finance system reports from a phone. One indicator per screen, a few kilobytes, and if the signal drops they reload and carry on."

Time	Screen	Say
0:15	Tap through, submit	"Same data as the desktop path, same audit trail."
0:25	Extractions, source cells visible	"On the desktop side they upload the template they already wrote. We read it and show every value beside the cell it came from. Nothing counts until a named human confirms it."
0:40	Portfolio, ranked by risk	"The department opens one screen and knows where to look."
0:50	Risk panel expanded	"And it tells you why. Late three quarters running. Sixty percent of the year gone, twenty percent of targets started. Seventy percent of the money drawn. A repeat audit finding. Five signals, five weights, adds to the score. Not a black box."
1:10	Unit cost	"Which lets you ask the real question. Not <i>did they report</i> . Was the money worth it. "
1:25	Citizen page	"And when DSAC decides to publish, a citizen sees what their money bought."

Closing line: built on open-source components, standard Java and PostgreSQL so the skills are available locally, deployable on-premise or in any cloud with data resident in South Africa, no per-seat licence as the entity count grows, and the department can take the codebase in-house.

3. Fix only what breaks on the demo path. A bug on a screen the judges will not see is not a bug this weekend. Write it down, leave it.

4. Rehearse the five answers.

Question	Answer
Where does the data come from?	Real published annual reports and APPs. Name the entities.
Does the AI decide anything?	No. Parsed values sit in a separate table with their source cell until a named person confirms them. The separation is in the schema. The risk engine is a weighted sum of five observable facts — nothing trained.

Question	Answer
Is this a new reporting burden?	No. The template digitises what entities already compile. It does not add a sixth report to the five they file.
Can you compare cost per outcome across entities?	No, and we do not. Each entity against its own plan. Peer comparison only within a sector, labelled indicative. There is no API parameter that would allow otherwise.
Why should the state trust this long-term?	Open-source stack, standard skills, any cloud or on-premise, no lock-in, the department owns the code and the data.

If something breaks live: stop, say "let me show you the recording", play it. Do not debug in front of judges. You lose thirty seconds and no credibility; debugging costs you both.

H44-H48 · Buffer and present

Sleep if the schedule allows. Arrive early. Test the room's projector and network before you need them. Two people present: one driving, one talking, both rehearsed.

7. Stack notes and traps

The five that cost hours

Flyway owns the schema, Hibernate only checks it. `ddl-auto: validate` means the application refuses to start when an entity and the database disagree. That is deliberate — it fails loudly at startup rather than silently at 3am. Fix the entity, or add a migration. **Do not switch to `update` to make the error go away;** you lose the discipline that makes this debuggable under pressure.

Never edit a migration that has already run. Flyway checksums it. Editing `V1__baseline.sql` after it applied gives a checksum error on next start. Add `V2__`, or during a hackathon just drop and recreate:

```
docker compose down -v && docker compose up -d
```

Custom claims are cached in the token. Change a user's role and nothing happens until they sign out and back in. This catches everyone once. Put it in the team chat at H3.

open-in-view is off. Lazy associations outside a transaction throw. Fetch what you need inside the service layer, or map to a DTO before returning. It is off on purpose: leaving it on hides N+1 queries that would surface under demo load.

Apache POI reads formulas two ways. `DataFormatter` with a `FormulaEvaluator` computes them; without one it returns the cached value. The parser falls back to cached when evaluation throws, which matters because the template's Variance column is a formula and a file saved by some tools carries no cached value.

Conventions used throughout

Entity:

```
@Entity
@Table(name = "target_result")
public class TargetResult {
    @Id @GeneratedValue private UUID id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "target_id")
    private Target target;

    @Enumerated(EnumType.STRING)
    private Enums.TargetStatus status;
}
```

Enums are persisted `STRING`, never `ORDINAL`. Ordinal storage means inserting a value into the middle of an enum silently reinterprets every existing row.

Migration: plain SQL under `src/main/resources/db/migration`, named `V1__baseline.sql`. Numbers ascend, names never change once applied.

Money: `numeric(18,2)` and `BigDecimal`. Never `double`. Rand figures in the hundreds of millions divided by target counts is exactly where floating point embarrasses you in front of a finance person.

Time: `Instant` for events, `LocalDate` for deadlines. The scheduler and the lateness calculation both pin `Africa/Johannesburg` explicitly rather than trusting the server default.

The one honest limitation

The project has **not been compiled end to end** — it was written in an environment that could not reach Maven Central. The risk engine is genuinely tested: ten tests, all passing, run against real Java, because that is the component a judge will interrogate.

Everything else is written carefully against Spring and JPA conventions but expect import and signature fixes on the first `mvn compile`. **Budget thirty minutes at H0**, before anything is built on top of it.

8. Traceability

What earns each judging criterion, and where it lives.

#	Criterion	Weight	Requirements	Evidence
1	Relevance	20%	FR-1 ... FR-5	All five mandatory requirements built. Anything descoped is presented with a concrete path, which the criteria explicitly allow.
2	Innovation	15%	FR-6, FR-8, FR-10	A machine-readable reporting standard where none exists. Template parsing, behavioural risk prediction, low-bandwidth submission.
3	Technical feasibility	20%	FR-6, FR-7, §5	A working end-to-end path, not slides. Submission flows to dashboard to citizen view in the demo.
4	UX and inclusivity	10%	FR-10, FR-11, NFR-5	Sub-100KB surfaces, one indicator per screen, connection-tolerant, plain language. A working channel, not a font-size claim.

#	Criterion	Weight	Requirements	Evidence
5	Data and insight	10%	FR-8, FR-9, NFR-4	Unit cost planned versus actual. Risk scores shown with contributing signals. Every figure traceable to a source cell.
6	Security and governance	10%	NFR-1, NFR-2, NFR-3	Four roles, POPIA-aligned, append-only performance data, human-in-the-loop enforced by schema, deterministic scoring.
7	Scalability and sovereignty	10%	NFR-7, NFR-8	Open-source stack, local skills, any cloud or on-premise, no lock-in, standard reusable by any department.
8	Presentation	5%	—	The 90-second thread, rehearsed three times and recorded as backup.

Where the marks actually are

Only 20 percent is tied to the specific challenge statement. Another 45 percent — innovation, data and insight, security, sovereignty — rewards how you build and how you explain it, and is available to any team that thinks about it. Most will not.

The implication for the schedule: criteria 1 and 3 together are 40 percent and both need things that visibly function, which is why day one is the mandatory floor and nothing else. Then spend what remains on explainability and the citizen view — cheap to build, disproportionately weighted.